

FMCW Radar Firmware Guide

CubeIDE build, flash, and source-code map

Generated for the STM32F429 FMCW radar firmware

2026-05-19

1. What You Have

This firmware is for an STM32F429VETx board that captures FMCW radar ADC samples and streams them to a PC over USB High Speed.

The important hardware chain is:

```
ADF4158 ramp + RF chain
-> LTC1420 12-bit ADC
-> STM32 DCMI peripheral
-> DMA2 Stream 1 double-buffer capture
-> SRAM frame ring
-> USB OTG HS bulk IN endpoint
-> Host PC
```

The CubeMX project at C:\Users\karun\OneDrive\Desktop\FMCWRadarFirmware was used as the pin map source.

The actual firmware you should build is in:

C:\Users\karun\Downloads\FMCWCode\firmware

2. Can You Use STM32CubeIDE?

Yes. You can open the firmware folder in CubeIDE.

This repo is not a normal CubeMX-generated project where CubeMX owns main.c. It is a hand-written firmware project that uses STM32 HAL/CMSIS, plus a small Makefile wrapper so CubeIDE can build it.

Do not press “Generate Code” from CubeMX into this folder. CubeMX would overwrite important architecture choices.

The important CubeMX choices that were intentionally overridden are:

CubeMX generated	Firmware uses	Why
180 MHz SYSCLK	168 MHz SYSCLK	Keeps TIM9 alignment math simple and matched to the architecture

CubeMX generated	Firmware uses	Why
DCMI HSYNC low	DCMI HSYNC high	PA4 is held high after alignment
DCMI falling PIXCLK	Falling PIXCLK	Updated per board bring-up request
PE5 as GPIO input	PE5 as TIM9_CH1 AF3	Needed to align capture to ADF4158 MUXOUT
USB DMA disabled	USB DMA enabled	Needed for high-speed streaming

3. The File You Build and Flash

Build from:

```
C:\Users\karun\Downloads\FMCWCode\firmware
CubeIDE or make will produce:
C:\Users\karun\Downloads\FMCWCode\build\firmware-cubeide\fmcw_radar_f429.elf
C:\Users\karun\Downloads\FMCWCode\build\firmware-cubeide\fmcw_radar_f429.bin
C:\Users\karun\Downloads\FMCWCode\build\firmware-cubeide\fmcw_radar_f429.hex
For CubeIDE debug/flash, use the .elf file:
build\firmware-cubeide\fmcw_radar_f429.elf
```

The ELF contains both the program and debug symbols, so CubeIDE can step through source code if you create a debug configuration.

4. CubeIDE Build Steps

Option A: Build After Importing the Firmware Folder

1. Open STM32CubeIDE.
2. Use File -> Import.
3. Choose an existing C/C++ project or Makefile project.
4. Select:

```
C:\Users\karun\Downloads\FMCWCode\firmware
```

5. Build the project.

The `firmware\Makefile` runs CMake/Ninja behind the scenes and writes output to `build\firmware-cubeide`.

Option B: Build From CubeIDE Terminal

In a CubeIDE terminal, run:

```
cd C:\Users\karun\Downloads\FMCWCode\firmware
make all
```

To flash with ST-LINK from the wrapper:

```
make flash
```

The flash target uses STM32CubeProgrammer CLI and writes the ELF through SWD.

5. Folder Map

Path	Purpose
<code>firmware\Makefile</code>	CubeIDE-friendly wrapper. Runs CMake configure/build and optional flash.
<code>firmware\CMakeLists.txt</code>	The real firmware build recipe: source files, HAL files, include paths, linker script.
<code>firmware\cmake\arm-none-eabi-gcc.cmake</code>	CMake toolchain file for ARM GCC.
<code>firmware\linker\STM32F429VETx_FLASH.ld</code>	Linker memory map. Places code in Flash and buffers in SRAM.
<code>firmware\startup\startup_stm32f429xx.s</code>	Reset vector and interrupt vector table startup code.
<code>firmware\Core\Inc</code>	Firmware header files.
<code>firmware\Core\Src</code>	Firmware C source files.
<code>host</code>	Python PC-side capture/parser tools.
<code>tests</code>	Host protocol tests.
<code>docs</code>	Bring-up and board configuration notes.

6. Important Header Files

File	What it contains
<code>Core\Inc\main.h</code>	Minimal global include for STM32 HAL.

File	What it contains
Core\Inc\fmcw_board_config.h	Board-specific pins and build-time options. Start here when checking hardware mapping.
Core\Inc\fmcw_protocol.h	USB frame format: magic number, header size, payload size, sample rate, flags.
Core\Inc\fmcw_capture.h	Capture buffer sizes and capture-state function declarations.
Core\Inc\fmcw_hardware.h	Hardware initialization and TX standby function declarations.
Core\Inc\fmcw_adf4158.h	Optional ADF4158 SPI programming API.
Core\Inc\usb_stream.h	USB streaming API.
Core\Inc\usb_vendor_stream.h	Vendor-specific USB class declarations.
Core\Inc\stm32f4xx_hal_conf.h	Enables the HAL modules used by this firmware.

7. Important Source Files

Core\Src\main.c

This is the application entry point.

It does this in order:

1. Calls HAL_Init().
2. Configures clocks with fmcw_clock_config().
3. Starts the microsecond timestamp system.
4. Initializes capture state.
5. Configures GPIO pins.
6. Resets the USB3317 PHY.
7. Initializes DCMI, DMA, USB, TIM9, and interrupts.
8. Optionally waits for USB enumeration.
9. Optionally programs the ADF4158.
10. Starts TIM9 alignment.
11. Runs the forever loop.

The forever loop calls:

```
fmcw_tx_service();
usb_stream_poll();
fmcw_capture_watchdog_poll();
```

So the main loop keeps TX enable updated, feeds USB frames to the host, and checks that ADF4158 MUXOUT pulses are still arriving.

Core\Src\fmcw_hardware.c

This file owns direct hardware setup.

Important functions:

Function	What it does
<code>fmcw_clock_config()</code>	Sets HSE bypass, PLL, bus clocks, and $MCO1 = HSE / 2 = 10$ MHz.
<code>fmcw_gpio_init()</code>	Applies the CubeMX pin map, sets safe output levels, and samples TX_STDBY.
<code>fmcw_usb_phy_reset()</code>	Pulses USB3317 reset on PE13.
<code>fmcw_tx_service()</code>	Reads TX_STDBY on PE11. If high, sets MIXER_EN PD13 and PA_EN PE0 high.
<code>fmcw_dcmi_init()</code>	Configures DCMI for 12-bit continuous capture.
<code>fmcw_dma2_stream1_init()</code>	Configures DMA2 Stream 1 Channel 1 double-buffer capture.
<code>fmcw_tim9_init()</code>	Configures TIM9 to watch ADF4158 MUXOUT on PE5.
<code>fmcw_nvic_init()</code>	Enables DMA, DCMI, USB, and TIM9 interrupts.

Core\Src\fmcw_capture.c

This file owns the capture buffer ring.

The firmware has four frame buffers. DMA writes into two of them using hardware double-buffer mode. The other two are used as spare/USB buffers.

When DMA finishes one slope:

1. The DMA interrupt calls `fmcw_capture_dma_tc_isr()`.
2. The completed buffer gets a 64-byte frame header.
3. The buffer is queued for USB.
4. A free buffer is attached back to DMA.
5. The slope sequence number increments.

Core\Src\stm32f4xx_it.c

This file contains interrupt handlers.

Interrupt	What happens
<code>DMA2_Stream1_IRQHandler</code>	One slope buffer has completed, or DMA fault occurred.
<code>DCMI_IRQHandler</code>	DCMI overrun/sync errors become frame flags.
<code>TIM1_BRK_TIM9_IRQHandler</code>	First MUXOUT starts aligned capture; later MUXOUT edges update heartbeat count.

Interrupt	What happens
OTG_HS_IRQHandler	USB HAL interrupt handler.
SysTick_Handler	HAL tick.
Core\Src\usb_stream.c	
This file moves completed frame buffers to USB.	
It checks whether USB is configured and whether EP1 IN is idle. If a frame is ready, it starts one bulk IN transfer. When USB reports completion, the buffer is returned to the free pool.	
Core\Src\usbd_vendor_stream.c	
This is the vendor-specific USB class.	
It creates one interface and one bulk IN endpoint:	
Endpoint: 0x81 Direction: IN, device to host High-speed packet size: 512 bytes	
Core\Src\fmcw_frame.c	
This writes the 64-byte frame header before each ADC payload.	
The host uses the header to read:	
• frame magic • protocol version • slope sequence • triangle sequence • slope direction id • error flags • sample rate • timestamp • MUXOUT count	
Core\Src\fmcw_adf4158.c	
This is optional ADF4158 SPI programming support.	
Right now autoprogramming is disabled:	
<code>#define FMCW_ENABLE_ADF4158_AUTOPROGRAM 0u</code>	
That is intentional because the real ADF4158 register values for center frequency and ramp settings must be filled in first.	

8. Board Pin Summary

TX Control

Signal	Pin	Behavior
TX_STDBY	PE11 input	Active high TX-enable request
MIXER_EN	PD13 output	High when TX_STDBY is high
PA_EN	PE0 output	High when TX_STDBY is high

If TX_STDBY is normally high on your board, the firmware starts transmitting after GPIO initialization.

ADF4158

Signal	Pin
PLL_CLK / SPI4_SCK	PE2
PLL_DATA / SPI4_MOSI	PE6
PLL_LE	PA1
PLL_CE	PE1
PLL_MUXOUT	PE5 / TIM9_CH1

DCMI ADC Bus

Signal	Pin
D0	PA9
D1	PC7
D2	PC8
D3	PC9
D4	PC11
D5	PD3
D6	PB8
D7	PB9
D8	PC10
D9	PC12
D10	PD6
D11	PD2
PIXCLK	PA6
HSYNC	PA4, driven by firmware
HSYNC feedback	PA7, routed from PA4 on the PCB
VSYNC	PB7, expected tied low

9. How Capture Starts

At boot, DCMI is configured but not capturing yet.

TIM9 watches MUXOUT from the ADF4158 on PE5. On the first MUXOUT edge, TIM9 waits a small delay for the LTC1420 pipeline offset. Then the TIM9 interrupt drives PA4 HSYNC high, checks the routed PA7 feedback signal, and sets the DCMI capture bit.

After that, HSYNC stays high. The firmware does not toggle HSYNC every slope.

One DMA buffer equals one slope:

```
10,000 samples per slope
2 samples packed per 32-bit DCMI word
5,000 DMA words per slope
20,000 payload bytes per slope
64 header bytes per USB frame
20,064 total bytes per frame
```

10. What To Check On The Board

Start with these checks:

1. PA8 should output 10 MHz MCO1.
2. USB3317 should have its 26 MHz reference and ULPI 60 MHz clock.
3. USB should enumerate as a vendor-specific device.
4. PE11 TX_STDBY high should make PD13 MIXER_EN and PE0 PA_EN go high.
5. PE5 should receive ADF4158 MUXOUT pulses every slope.
6. PA4 HSYNC should go high after the first MUXOUT alignment event.
7. PA7 should also read high because it is the routed-back HSYNC feedback signal.
8. Host capture should receive frames with valid magic and increasing slope sequence.

11. Common Beginner Mistakes

Do not edit generated files in the CubeMX folder and expect this firmware to change. The active firmware is under C:\Users\karun\Downloads\FMCWCode\firmware.

Do not regenerate CubeMX code into this folder unless you are ready to manually re-merge the architecture.

Do not flash the CubeMX-generated empty application from

C:\Users\karun\OneDrive\Desktop\FMCWRadarFirmware. It configures pins, but it does not implement the radar pipeline.

Do not flash the .bin from an arbitrary address unless you know the flash base address. In CubeIDE, use the .elf.

12. Quick Reference

Build:

```
cd C:\Users\karun\Downloads\FMCWCode\firmware
make all
```

Flash through ST-LINK:

```
make flash
```

ELF to use in CubeIDE debug configuration:

```
C:\Users\karun\Downloads\FMCWCode\build\firmware-cubeide\fmcw_radar_f429.elf
```

Most important source files to read first:

```
firmware\Core\Src\main.c
firmware\Core\Src\fmcw_hardware.c
firmware\Core\Src\fmcw_capture.c
firmware\Core\Src\stm32f4xx_it.c
firmware\Core\Inc\fmcw_board_config.h
```